

Rail-Crossing Violation Warning (RCVW) Software API Description

Version 4.0

Battelle
505 King Avenue
Columbus, Ohio 43201

January 13, 2025

Table of Contents

	Page
Table of Contents.....	2
Chapter 1. Overview.....	3
Chapter 2. High-Level Architecture.....	4
Chapter 3. Component Design	5
RCVW Messaging Core Software	7
Apache Kafka.....	7
AMQP	7
Asynchronous I/O (AIO)	7
Simple Network Management Protocol (SNMP)	8
Global Positioning System Daemon (GPSd).....	8
VBS Software	8
Message Receiver Plug-in.....	9
Differential Global Positioning System (GPS) Plug-in	9
RCVW Plug-in	9
Human-Machine Interface (HMI) Plug-in.....	9
RBS Software	9
MAP Plug-in	10
HRI Status Plug-in.....	10
RTCM Plug-in.....	10
RSU Immediate Forward Plug-in	10
CBS Software	11
Azure Function App(s).....	12
Azure Container App(s)	16
Azure Data Log	17
Chapter 4. Code.....	20
Building	20
RBS Component	20
VBS Component.....	20
CBS Components.....	20
Execution	21
Appendix A: CBS Resources	28

Chapter 1. Overview

The Rail-Crossing Violation Warning (RCVW) is a prototype Vehicle-to-Everything (V2X) software application meant to provide improved situational awareness and alerting for road users concerning the state of active railroad monitoring systems. V2X software applications provided by the US Department of Transportation (USDOT) Federal Highway Administration (FHWA) and Federal Railroad Administration (FRA) are built upon a reference implementation known as V2X-Hub¹. The V2X-Hub is itself a collection of intercommunicating software components for an Intelligent Transportation System (ITS) that uses Battelle's Transportation Message eXchange (TMX[®]) platform.

The TMX[®] software architecture provides a robust, flexible, scalable, and extensible interface allowing multiple connected vehicle (CV) applications to co-exist in a single platform, while sharing critical resources between applications. TMX[®] is hardware-agnostic and can be modified to work with current roadside unit (RSU) or after market on board unit (OBU) hardware to facilitate CV message transmission and receipt.

The critical operations of OBUs and RSUs include sending and receiving SAE J2735 messages over the Cellular Vehicle-to-Everything (C-V2X) wireless protocols, and many CV applications require interfaces to external systems such as Global Navigation Satellite System (GNSS) positioning information, vehicle controller area network (CAN) data, traffic signals², and cloud services. The TMX[®] middleware provides CV applications with the ability to exchange messages and external data efficiently and effectively across software components using a publish-subscribe model. The critical communication functions required for developing comprehensive safety and mobility applications are all implemented natively within the TMX[®] platform, leaving the application programmers to focus on coding the *business logic*. This ultimately leads to rapid, productive development of CV safety and mobility applications.

The goals of the TMX[®] platform and its V2X-Hub derivative is to provide:

- A ubiquitous means of message Tx and Rx over wired and wireless transport, including the licensed C-V2X bands and unlicensed spectrum such as cellular and Wi-Fi.
- A built-in means for efficient encoding and decoding of critical information without the need for custom data structures and code generation.
- A prescribed component architecture to maximize scalability and reusability.
- Native support for common CV and ITS functions such as simple network management protocol (SNMP), CAN, SAE J2735, National Transportation Communications for ITS Protocol (NTCIP), Global Navigation Satellite System (GNSS), and specialized functions such as geofencing and driver alerting.

The FHWA maintains a version of V2X-Hub that was originally built on the release 3.0 of TMX[®], which is limited to a singular custom communication protocol built specifically for the US DOT prototype projects. While new components have been added to V2X-Hub, the underlying platform was never enhanced to take advantage of compiler evolution, or to include additional communication protocols and associated performance improvements. In order to meet the requirements of the RCVW Phase III project, Battelle has adapted the FHWA release of the

¹ <https://github.com/usdot-fhwa-OPS/V2X-Hub>

² TMX[®] works on NTCIP-compliant traffic signal controllers.

TMX[®] architecture to be neutral to the messaging core and ported the necessary V2X-Hub components. This document details the updated TMX[®] architecture and the overall RCVW software design.

Chapter 2. High-Level Architecture

Battelle has developed TMX[®] to be interoperable with a large range of hardware specifications, and to accommodate the plug-ins and applications that represent the information needed to produce and transmit/receive CV application messages. Figure 1 below illustrates the basic tiered architecture for development of a V2X ecosystem built using the TMX[®] platform.

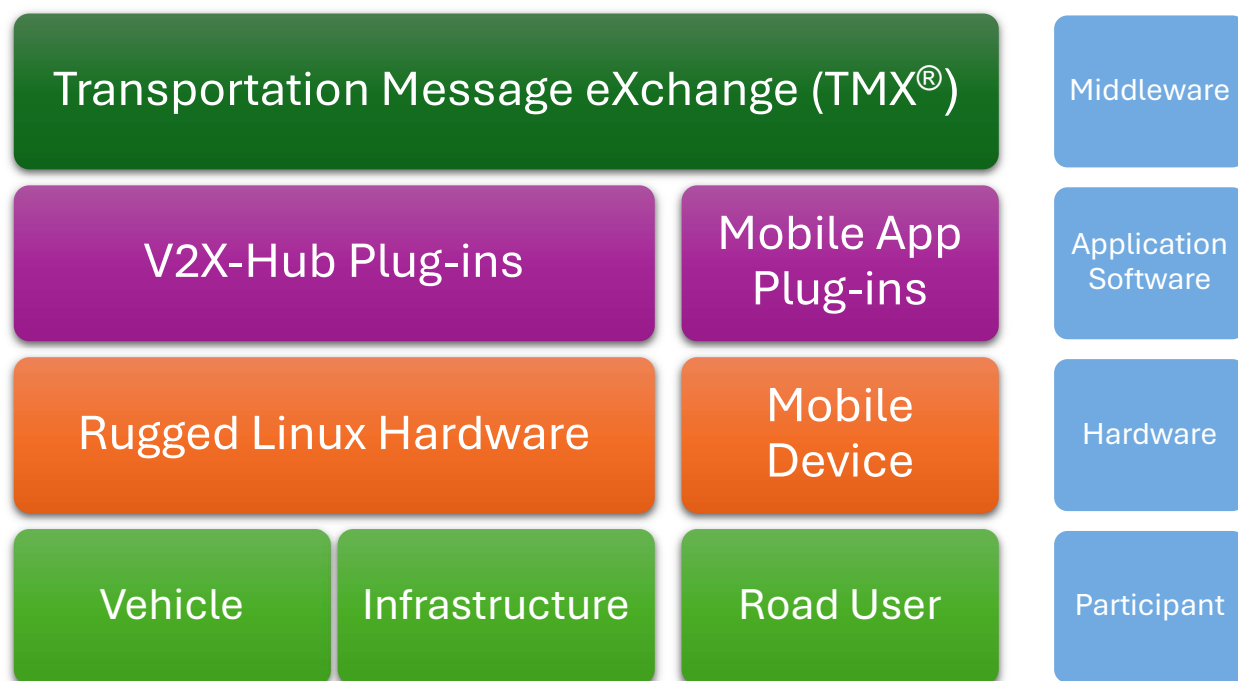


Figure 1 – TMX[®] V2X Architecture

Within the TMX[®] architecture, each software application or service is set up as a participant of the message exchange, dubbed a plug-in. This architecture is optimized for rapid development of the application software, i.e. the business logic, as the basic message exchange platform abstracts the complexities of message encoding and transmission as well as facilitating cooperation and data sharing amongst the various components. This includes data exchange between plug-ins specifically for in-vehicle, roadside infrastructure, or other road users such as pedestrians or bicyclists, plus any supporting software required for each.

The communication of messages between plug-ins occurs using the Publish/Subscribe (Pub/Sub) architecture. Some plug-ins produce messages while others consume them, but all messages are routed through some message broker, known as the TMX[®] Messaging Core. See [Figure 2](#) for the general message flow through TMX[®].

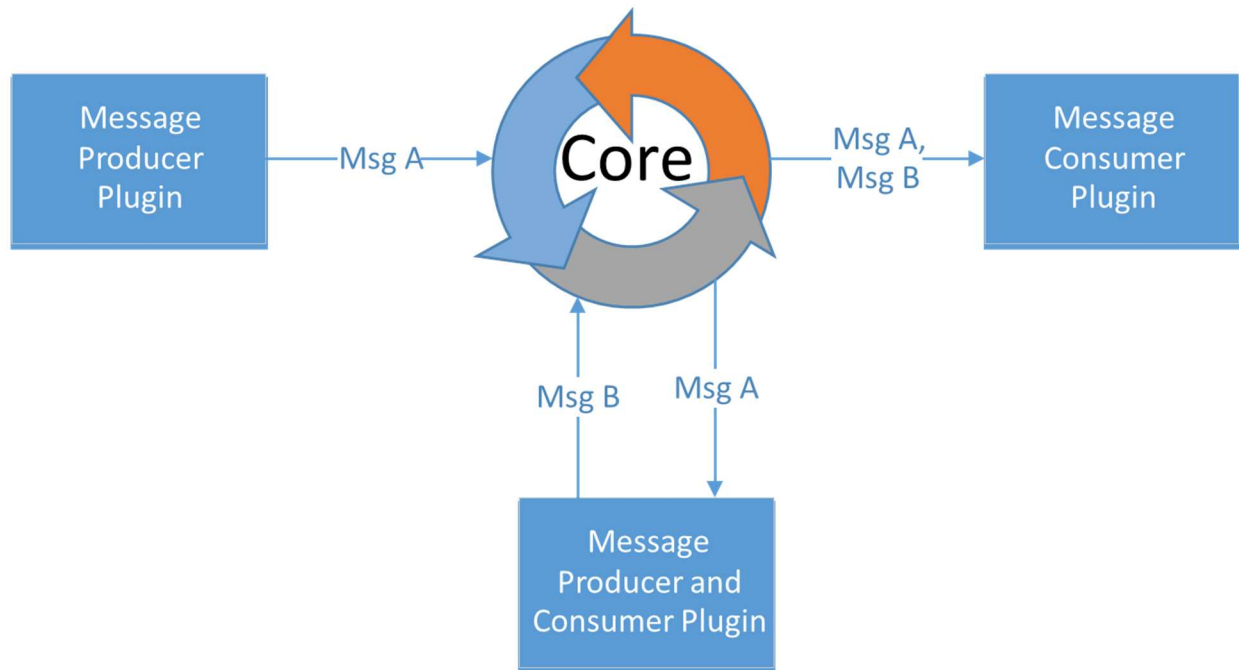


Figure 2 – The TMX™ Publish/Subscribe Architecture

Previous versions of TMX® generally supported a single core messaging broker per device, even if the plug-ins could operate on remote systems. Additionally, this broker was a custom prototype built for the purpose of the V2X Hub project demonstration. While this architecture functions adequately, far more capable software now exists which performs more advanced features. Therefore, the purpose of the architecture update is to integrate the TMX® plug-ins across a wide range of cloud-native message brokers that operate independently of the V2X specific applications. In other words, TMX® 4.0 is a refresh of the architecture meant to decouple the Core from the plug-ins. The RCVW III project was used to develop the necessary architectural improvements as well as the porting of plug-in functionality to operate under that new architecture. This includes the full RCVW system plug-ins as well as a subset of the V2X Hub plug-ins. It is highly recommended that a complete migration of the V2X-Hub plug-ins to the new architecture be accomplished either through an additional FHWA funded effort or via community support.

Chapter 3. Component Design

Effectively, the TMX® software is middleware that provides an Application Programming Interface (API) to facilitate the rapid development of the applications. While conceptually, the goal is to build a complete V2X application, this is accomplished by building and combining plug-ins. For example, consider the three basic components of the RCVW application:

1. **Vehicle-Based Subsystem (VBS):** Responsible for determining unsafe conditions and issuing alerts based on the RBS information and vehicle specific telemetry.
2. **Roadside-Based Subsystem (RBS):** Responsible for reporting the geometry and pre-emption status of the highway-rail grade crossing.
3. **Cloud-Based Subsystem (CBS):** Responsible for disseminating rail-crossing information to other subscribers.

The required plug-ins for each of these components, along with the data exchange between them, is shown in [Figure 3](#).

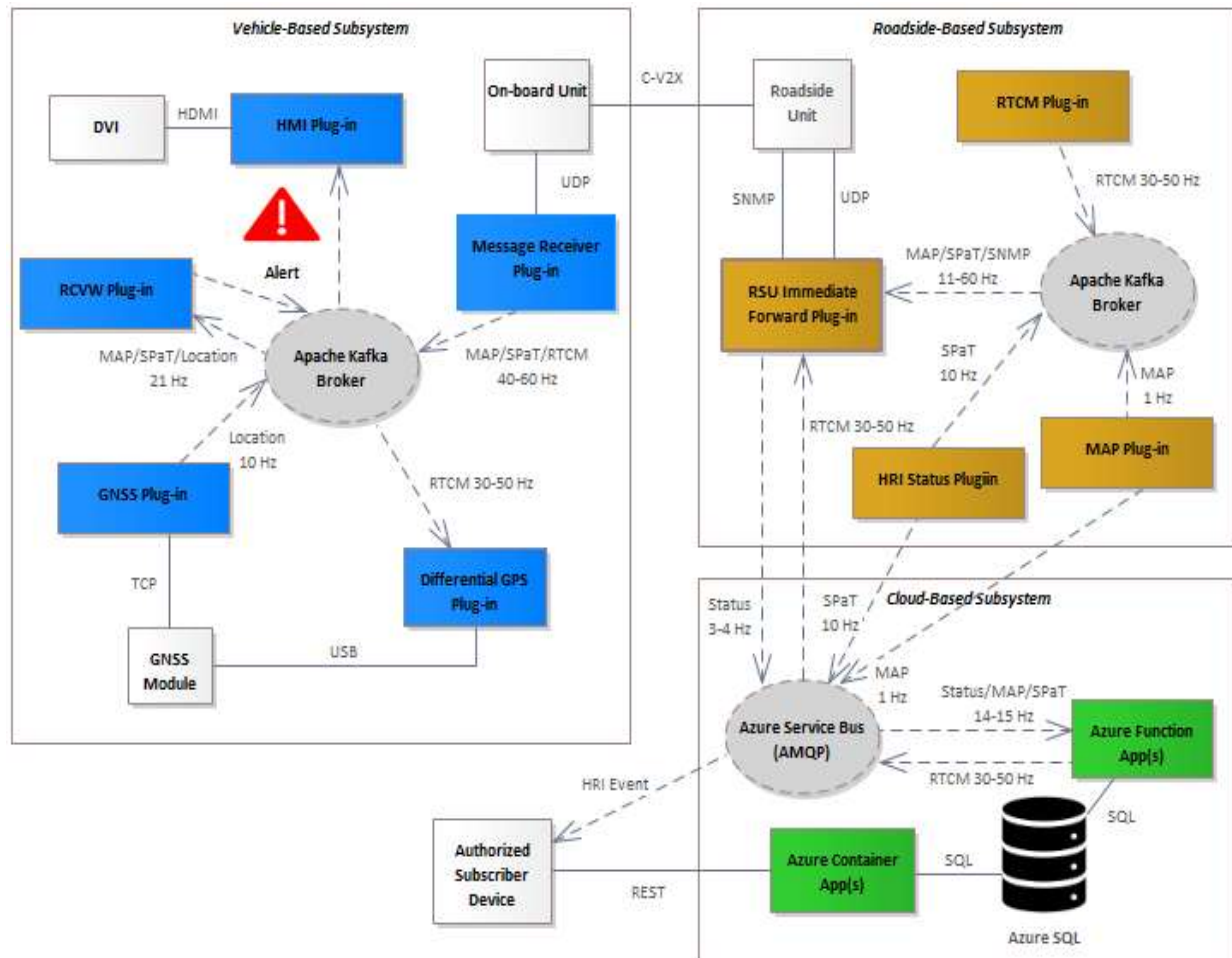


Figure 3 – RCVW Software Component Architecture

RCVW Messaging Core Software

The core message exchange within RCVW is provided by TMX® in order to connect all the application plug-ins together. The primary driver for the TMX® v4.0 architecture is to decouple the core messaging protocol from the plug-ins, and to support multiple different brokers simultaneously. This is key for RCVW application because the roadside component for Phase III was designed to send its messages both through the local radio and to the cloud-based component.

While latency of the message delivery is critical for optimal performance, there are a number of other factors such as scalability that may guide specific implementations at different junctions of the system or for particular applications. Therefore, every plug-in must be unaware of the underlying technology implemented in the core it is connected with. Likewise, a plug-in may be connected to multiple cores at once in order to publish to multiple exchanges with different scalability needs. The only real requirement of the messaging core technology is it is capable of some basic asynchronous Pub/Sub messaging. This section lists some messaging or general communication technologies that have been adapted to operate within TMX® for the RCVW project.

Apache Kafka

After previous research on general-purpose Internet of Things (IoT) messaging protocols determined that Apache Kafka provides very low latency with minimal overhead and a simple to use C++ API, the Battelle team decided that Apache Kafka would be the default opensource broker for TMX® applications, particularly for the internal routing of messages within a subsystem. Therefore, this means that a separate Kafka broker service is required for both the RBS and the VBS, which is provided by the default RCVW Docker deployment.

AMQP

The Advanced Message Queuing Protocol (AMQP) is a standardized, distributed system similar in function to the Unix POSIX message queues. Whereas the Apache Kafka service is to be used for intra-subsystem messaging, the AMQP standard is already supported in a number of cloud services such as Microsoft Azure Service Bus, which provides a near real time solution for cloud ingest. The Apache Qpid Proton toolkit provides the underlying cloud enabled C++ implementation for the AMQP broker in TMX®. The RCVW RBS component software comes equipped with the ability to connect to any AMQP broker service, including the Service Bus, which must be administratively deployed to the CBS Azure instance.

Asynchronous I/O (AIO)

Although not explicitly depicted in [Figure 3](#) as a TMX® Core technology, the platform provides a broker implementation for access of file system and/or network resources using an asynchronous interface. These include Internet Protocol (IP) access to Transmission Control Protocol (TCP) or User Datagram Protocol (UDP) services, as well as data on serial interfaces such as the Universal Serial Bus (USB). While many libraries for asynchronous access to file and network resources exist already, this particular implementation adheres to the specific Pub/Sub architecture that a TMX® plug-in expects, and thus allows for the same kind of data handling. The main purpose of this message exchange within RCVW is to support the V2X-Hub default message receiver plug-in, which is used to ingest incoming SAE J2735 messages from the external on-board radio unit (OBU).

Simple Network Management Protocol (SNMP)

Many ITS systems use SNMP to manage devices, and V2X applications are no different. The SNMP broker is specifically used to pull management data off of National Transportation Communications for ITS Protocol (NTCIP) enabled devices. In the case of RCVW, the RSU radio is managed via SNMP like other NTCIP devices, and that is how one can obtain statistics on the current operational state. The TMX[®] SNMP broker software provides the general *get* and *set* operations to perform on any SNMP agent, but a plug-in must configure what data from the Management Information Base (MIB) to control. The RBS RSU Immediate Forward plug-in in RCVW uses this exchange to retrieve radio diagnostics from the external radio using the RSU 4.1 MIB, which are relayed through diagnostic messages to the CBS and monitored.

Global Positioning System Daemon (GPSd)³

The GPS is a service that monitors one or more Global Navigation Satellite System (GNSS) receivers attached to a host computer through serial ports, making all data on the location/course/velocity of the sensors available to be queried over a network interface. The TMX[®] GPSd broker software makes connections to this service in order to forward the incoming data stream to plugins. The software supports the typical connection that will stream time, position and velocity (PVT) data along with the satellite fix and position information. A National Marine Electronics Association (NMEA) reports the low-level NMEA *strings* that are outputs from the GNSS receivers. There is also support for raw messages from the receiver, including the NMEA strings and custom messages from each receiver manufacturer. Any plug-in can be configured to utilize these connections. Currently, this core messaging system only reads from the service, although writing to the device may be supported in the future. For RCVW, the VBS component requires the GPSd connection for attaining PVT data from the GNSS module. Additionally, however, the RBS component may also utilize this connection to receiving ground-based corrections from a locally connected base station.

Note that previous iterations of V2X-Hub included a separate Location plug-in that translated GPSd messages through the TMX[®] core messaging broker. However, since the 4.0 architecture allows for multiple simultaneous broker connections in each plug-in, it instead made more sense for RCVW III to build the location messaging into each plug-in that requires data from GPSd. Therefore, there is no longer an external Location plug-in.

VBS Software

Most of the safety logic is built into the plug-ins that run inside the vehicle. For this reason, there are some unique considerations for these. First, latency of the message delivery has a real impact on the operation of the plug-in. For example, every tenth of a second (100 milliseconds) wasted in transmission for a vehicle traveling at 50 mph directly implies 7.33 feet less of distance required to stop. Therefore, the messaging system should be optimized for the lowest latency, which means a kernel implemented loop-back interface for messages to be passed internally around the vehicle as opposed to traversing a serial network interface. Secondly, the asynchronous messages that are used to determine crossing state and vehicle telemetry must all be aligned to a common time frame based on the time created, and not received, to ensure data consistency.

³ <https://gpsd.gitlab.io/gpsd/>

Since one of the key benefits of the TMX[®] software is to be able to run on numerous different platforms, there is also a wide range of possible deployment hardware for vehicle plug-ins, including existing systems like C-V2X OBUs. However, it may be difficult to find OEM partners willing to run Battelle software on their equipment due to performance and sizing constraints. Therefore, the RCVW project also supplies VBS prototype hardware that is optimized for performance of the on-board safety system. It is meant to be used as a black box solution for demonstration without integration within a specific vehicle.

Message Receiver Plug-in

CV applications are primarily defined by the exchange of SAE J2735 messages over dedicated radio spectrum in the 5.9 GHz band. The V2X-Hub Message Receiver plug-in is designed to accept the J2735 payload on a UDP server and forward directly to a TMX[®] core instance, in this case the default Apache Kafka broker on the VBS. For convenience, the RCVW III prototype architecture depicts the Message Receiver Plug-in directly on the VBS computing platform, but it may also be placed on separate hardware as necessary, including co-located with the OBU radio if the hardware allows.

Differential Global Positioning System (GPS) Plug-in

This RCVW specific plug-in helps to improve the accuracy of the positioning data. This is done by injecting Radio Technical Commission on Maritime Services (RTCM) Real-Time Kinematics (RTK) correction messages directly into the VBS GNSS module over its USB interface. The correction messages are expected to be received over C-V2X radio communications, and while it is conceivable that the bytes be relayed through the *GPSd* server, this was declined in lieu of the more efficient and direct method of writing to the device. This plug-in requires special elevated permissions in the Docker deployment in order to write to the serial device driver.

RCVW Plug-in

Previous phases of the RCVW project focused on developing an effective algorithm for issuing rail-crossing informational and warning alerts to the driver. The RCVW plug-in implements this algorithm based on the position and speed of the vehicle, along with the geometry and state information from the RBS. Therefore, it consumes the highest number of messages from the local Apache Kafka broker, while only producing the alert messages that need to be displayed.

Human-Machine Interface (HMI) Plug-in

This plug-in drives the HDMI monitor or infotainment display on behalf of the RCVW plug-in by turning the specific alert into an image to present on the screen. While the operational displays were not significantly altered for this phase of the project, this code was modified to read messages directly from the VBS Apache Kafka broker. That ability is enable through the use of a third-party container⁴.

RBS Software

The job of the roadside plug-in is to communicate with ITS infrastructure devices, such as the rail-crossing controller system, and generate TMX[®] messages to pass to the vehicles over the

⁴ <https://hub.docker.com/r/stefanat/kafka2websocket>

radio and to external subscribers via the cloud-based component. For RCVW, the RBS must communicate the geometry and state of the Highway-Rail Intersection (HRI).

MAP Plug-in

The geometry of the HRI is summarized within a SAE J2735 MAP message, that lists the position of the intersection, and the approach lanes. For RCVW, the MAP is configured to include a tracked vehicle lane that exists in a distinct *signal group* from the vehicle approach lanes that it crosses. The V2X-Hub MAP plug-in provides the ability to specify the precise bytes (in hexadecimal) that should be broadcast every second for this HRI. Each MAP message is published to the local Apache Kafka broker and over AMQP to the Azure Service Bus (see [Figure 3](#)).

HRI Status Plug-in⁵

This RCVW specific plug-in reads the HRI preemption signal status and forms a J2735 SPaT message for broadcast 10 times per second. A SPaT message only contains the current state for each signal group at the HRI, thus both MAP and SPaT are needed to decode the current preemption state of the HRI tracked lane. When the preemption signal is active, the tracked lane signal group is in a *protected movement allowed* phase, similar to a green light, while the vehicle lanes are in a *stop and remain* phase, similar to a red light. When the preemption signal is off, the phases of these signal groups are reversed. Each SPaT message is published to the local Apache Kafka broker and over AMQP to the Azure Service Bus (see [Figure 3](#)).

RTCM Plug-in

The primary purpose of the V2X-Hub RTCM plug-in is to receive binary RTCM data and convert those to an SAE J2735 message that can be broadcast by the roadside radio. However, the RBS may be configured to receive RTCM data in various different ways⁶. While the plug-in itself can be used for directly connecting to a Network Transport of RTCM via IP (NTRIP) caster and pulling RTCM messages, this option also is dependent on known latitude and longitude information to receive boot-strap location information (NMEA). Additionally, previous RCVW iterations used the GPSd to pull RTCM data directly from a GNSS receiver configured to act as a local base station. While both of these options are functional, they also utilize additional RBS resources and network. Therefore, this plug-in can also be configured to receive RTCM data from the Azure Service Bus, which is supplied by the proxy server in the CBS, thus eliminating the need for the Location plug-in. In any case, the resulting J2735 messages are pushed along the local Apache Kafka broker to be broadcast over C-V2X. It is still up to the receiving vehicle to apply the corrections to its local GNSS receiver. This is, of course, done automatically by a VBS-equipped vehicle.

RSU Immediate Forward Plug-in

The RSU Immediate Forward is a V2X-Hub plug-in that receives messages from a core broker and pushes them to a connected RSU radio using the RSU 4.1 immediate forward specification. It is expected for RCVW that each MAP, SPaT, and RTCM message is to be broadcast,

⁵ More detailed information regarding the HRI pre-emption signal detection options can be found in the RCVW Architecture and Design Document

⁶ More detailed information regarding the position correction options can be found in the RCVW Architecture and Design Document

however the plug-in will only forward message types configured in the RSU immediate forward table. Others are dropped. Additionally, the radios are configured within the plug-in as SNMP agents so this plug-in can also monitor the radio performance. This information is published to the Azure Service Bus in the CBS via plugin diagnostic messages using AMQP (see [Figure 3](#)).

CBS Software

The cloud-based subsystem is not so much a TMX[®] subsystem as it is a Microsoft Azure subsystem that interacts with TMX[®] plug-ins through a common communication protocol. Therefore, the software is not developed using the standard TMX[®] libraries. This is an example of how a V2X ecosystem can be built using software that is not specifically built using V2X-Hub.

As depicted in the [Figure 3](#), the RCVW RBS subsystem exchanges messages with the CBS. For the Azure cloud, this is accomplished through the AMQP messaging core software (Apache Qpid Proton) talking through pre-defined topics on the Azure Service Bus. Although Apache Kafka is a supported protocol for data ingest to Azure Event Bus, the Battelle team determined that the low latency, raw streaming capabilities of Service Bus along with the software flexibility provided through AMQP was best suited for use in this project⁷. The senders and receivers that are built into the CBS, however, are all written in Python and therefore utilize the native Azure Service Bus Python software developer kit (SDK)⁸.

There are two basic methods for a subscriber to obtain data from the CBS. First, the polling method requires the subscriber to query directly from the CBS database. This is facilitated through the use of a RESTful API built into an application container deployed within Azure. While all key stored data within the CBS is available to be polled, it does require the subscriber to periodically execute queries in order to keep up to date with the fluctuating data, such as HRI pre-emption signal status and RBS operational status. Additionally, the web-based connectivity of such a poll makes it naturally slower, thus increasing potential latency concerns for certain applications of the data. Therefore, there is also a push method available, which lets the cloud components that detect the HRI pre-emption signal status as well as the RBS operational state, to forward resulting AMQP events through the Azure Service Bus. This is a highly efficient manner of distribution but has the drawback of potentially missing deliveries if the client is not available to receive them. Fortunately, those issues can generally be remedied by proper configuration of the Azure Service Bus, including long time to live values for the messages and ensured delivery options. However, even event driven applications also will require polling for general information about the HRI.

The cloud-based component of RCVW is built into two categories. The first are programs that execute upon triggers based on Azure events, such as receipt of incoming messages on the Service Bus. Each of these use the Function App capabilities in Microsoft Azure and are programmed in Python. Note that if no Service Bus messages are received, then an Azure Function App is never executed, thus making its resource usage highly efficient. Some software components, however, must be run at all times. Thus, the second cloud-based category of Azure Container Apps are used instead for those. A Container App generally are standalone components built based on another software tool, such as the Azure Data API Builder or programming language, and are constantly running.

⁷ See the *Cloud Access Performance Testing Document* technical memo

⁸ <https://learn.microsoft.com/en-us/python/api/overview/azure/servicebus-readme?view=azure-python>

Azure Function App(s)

The Microsoft Azure cloud services include applications that can automatically be triggered by events in the cloud, including messages arriving on the Azure Service Bus. The CBS provides some default Azure Function Apps that record incoming messages into an SQL database. This function also decodes the message using a portion of V2X-Hub software so that the HRI preemption signal information can be used. Other functions then are used to update the current signal state and, if necessary, generate an HRI event message that is put back on the Azure Service Bus for downstream consumption by authorized subscribers. The following is a description of the Azure function apps.

rcvw-cbs-azure-ingest-azure

This is an Azure Function App that is built into a Docker container and used for writing records to the RCVW CBS data log, which for Azure deployments is an Azure SQL Database. This Python function is automatically triggered when a message arrives at the Azure Service Bus subscription and includes a portion of compiled TMX[®] software for decoding the SAE J2735 messages.

A separate trigger exists for each subscription for the specific Azure Service Bus topic that this App is listening for, and the deployment of the App must scale based on the number of RBS systems the CBS must monitor. The `j2735.map rsu-incoming` subscription handles J2735 MAP messages from the Map Plug-in on any equipped RBS system. The `j2735.spat rsu-incoming` subscription handles J2735 SPaT messages from the HRI Status Plug-in on any equipped RBS. Likewise, there is a trigger for the `rsu-incoming` subscription of status and error topics for those plugins plus the RBS Immediate Forward Plug-in. Since all status and error messages are already JSON encoded, those require no additional decoding and are stored as-is in the Data Log.

Every incoming message is automatically written to the `RBS_INCOMING_MESSAGE` table, but the decoded MAP and SPaT messages are additionally written to `RBS_INCOMING_MAP` and `RBS_INCOMING_SPAT`, respectively. If the HRI includes a IEEE 1570 interface, a plugin status message containing the contents of the HRI rail crossing operational state message (9.4.40) is detected in the ingest process and written to the table `RBS_IEEE1570_4909_MESSAGE`.

rcvw-cbs-event-mgr-azure

This Azure Function App monitors the current state of the RBS. This includes checking that the RBS messages are being received at the appropriate rate, and independently detecting the current state of the HRI preemption signal. For convenience, the detection is done post-logging, thus may be lagging real-time by a few seconds. Additionally, each check is automatically scheduled by the running App, at first when a new RBS is detected, and subsequently every half second.

There are two scheduled checks. First, the RBS check is done to verify that:

1. All J2735 messages (MAP, SPaT, and RTCM) are being received at the expected rate.
2. All RBS plugins are reporting status at the expected rate.
3. All configured RSUs are operational and are currently sending radio broadcasts.

4. All expected IEEE-1570 heartbeat messages are being received, if applicable⁹

If any of these checks fail, then the `RBSOperational` status is set to `False (0)`. Only after all of these criteria are met will the `RBSOperational` status be set back to `True (1)`.

Secondly, a specific check for the HRI preemption signal status is done. If the RBS is in the operational state, then the HRI Active state will be set to:

1. True (1) if the `HRIActive` field was `False (0)` but now has preemption active
2. True (1) if the `HRIActive` field was `True (1)` and still has preemption active
3. False (0) if the `HRIActive` field was `True (1)` but now has preemption inactive
4. False (0) if the `HRIActive` field was `False (0)` and still has preemption inactive

The main purpose of this App is to send events to subscribers based on the above criteria. A CBS subscriber application must obtain an Azure Service Bus connection string and program its own client of the Service Bus using any of the supported AMQP client libraries, such as the available C# .Net or Python Azure libraries. A general topic called `cbs.events` is where every HRI event message is sent, but each subscriber application should use the HRI-filtered subscription for monitoring their specific HRI. For example, while the `rsu-outgoing` subscription will hear all HRI event messages, the `HRI_10785` subscription is specifically set up to hear only the events for HRI number 10785. Each RBS-equipped HRI will have its own topic subscription.

Every CBS message will have a JSON-encoded body (`contentType application/json`) for maximum flexibility of the data. The `subject` of the AMQP message will be the HRI ID (e.g., 10785) and there will be an additional custom property called `timestamp` that provides the Linux timestamp when the message was created (**not** received by Service Bus).

Any error detected in the RBS, whether through the event manager checks or directly incoming on any Plug-in error topic will result in an event message body in the form of:

```
{
  "HRI": 10785,
  "code": 121,
  "message": "RBS J2735 MAP rate dropped below the minimal operational threshold of 8
messages per second"
}
```

The `HRI` field denotes the crossing and should be identical to the message `subject`. The `code` is a numeric error code, and the `message` is a summary of the problem. There may also be an `exception` field included if the error was a result of a program run-time error. Note that a `code` of 0 indicates that this message is informational, and no error actually occurred. For example:

⁹ The IEEE-1570 connectivity is optional for RCVW (see the requirements), but provides the ability to detect connectivity issues between the RBS and the train detection system.

```
{
  "HRI": 10785,
  "code": 120,
  "message": "RBS J2735 message rates restored above minimum operational thresholds"
}
```

The HRI Active event is not an error message at all, thus contains no `code`, but simply shows the preemption state transition from Off to On or vice versa. For example:

```
{
  "HRI": 10785,
  "code": 1,
  "active": true,
  "message": "Preemption signal activated"
}
```

Or

```
{
  "HRI": 10785,
  "code": 0,
  "active": false,
  "message": "Preemption signal deactivated"
}
```

Table 1 below describes each of the possible HRI event codes.

Table 1: HRI Event Codes

HRI Event Code	Meaning
0	HRI pre-emption signal was deactivated. This event will be sent when the signal transitions from on to off and additionally when the RBS operational state is restored at a time when the signal is not active. The <code>active</code> flag is also set to <code>false</code> .
1	HRI pre-emption signal was activated. This event will be sent when the signal transitions from off to on and additionally when the RBS operational state is restored at a time when the signal is active. The <code>active</code> flag is also set to <code>true</code> .
1XX	Some change occurred in the expected messaging. These are in order of priority, meaning only the lowest value (highest priority) notice will be issued.
110	J2735 SPaT messaging rates that were below the operational thresholds have been restored above the minimum.
111	J2735 SPaT messaging rates that were above the operational thresholds have dropped below the minimum.

120	J2735 MAP messaging rates that were below the operational thresholds have been restored above the minimum.
121	J2735 MAP messaging rates that were above the operational thresholds have dropped below the minimum
130	J2735 RTCM messaging rates that were below the operational thresholds have been restored above the minimum.
131	J2735 RTCM messaging rates that were above the operational thresholds have dropped below the minimum
160	RBS diagnostic messaging that was absent from the HRI status plug-in has been restored.
161	RBS diagnostic messaging that was being received from the HRI status plug-in is now absent.
170	RBS diagnostic messaging that was absent from the MAP plug-in has been restored.
171	RBS diagnostic messaging that was being received from the MAP plug-in is now absent.
180	RBS diagnostic messaging that was absent from the RSU plug-in has been restored.
181	RBS diagnostic messaging that was being received from the RSU plug-in is now absent.
190	RBS diagnostic messaging that was absent from the RTCM plug-in has been restored.
191	RBS diagnostic messaging that was being received from the RTCM plug-in is now absent.
2XX	Some change occurred at the RSU radio. These are in order of priority, meaning only the lowest value (highest priority) notice will be issued.
210	The RSU clock that was stopped is increasing. This likely denotes that the RSU is back on-line. The RSU host information will also be supplied in the event, if available.
211	The RSU clock that was increasing has now stopped. This likely denotes that the RSU has gone down. The RSU host information will also be supplied in the event, if available.
220	The RSU mode state that was non-operational is now operational. This likely denotes that the RSU was taken out of stand-by mode. The RSU host information will also be supplied in the event, if available.
221	The RSU mode state that was operation is now non-operational. This likely denotes that the RSU was manually put into stand-by mode. The RSU host information will also be supplied in the event, if available.
250	The RSU transmission rate that was below the operational threshold has been restored above the minimum. This likely denotes recovery from transmission error. The RSU host information will also be supplied in the event, if available.
251	The RSU transmission rate that was above the operational threshold has dropped below the minimum. This likely denotes some transmission error. The RSU host information will also be supplied in the event, if available.
3XX	Some change occurred in the train detection system status. Note that this currently is only available if the HRI is equipped with IEEE-1570 connectivity. These are in order of priority, meaning only the lowest value (highest priority) notice will be issued.
310	The train detection system that was disconnected is now actively connected.
311	The train detection system that was actively connected is now disconnected.

Azure Container App(s)

Another deployment option in the Microsoft Azure environment is to create a custom Docker container and let the cloud system manage its operation and resource consumption. For RCVW, the primary Azure Container App is used to expose the SQL database tables as a RESTful API for consumption by authorized subscribers. This custom container utilizes the Azure Data API Builder (DAB) base container but also incorporates a custom authentication mechanism to ensure only subscribers can use the API. The following is a description of the Azure container apps.

rcvw-cbs-rtcm-azure

This Azure Container App is used for forwarding RTCM messages from the CORS network to the Azure Service Bus at an HRI specific subscription of the `v2x.rtc3` topic. Just like for the HRI events, the message `subject` should be the target HRI, thus the subscription should automatically filter the incoming messages by `HRI_ID` in order to ensure that the RTCM Plug-in on the correct RBS receives only those correction that pertains to its position. The RBS RTCM Plug-in must be configured in order to receive those messages at the proper subscription, convert them to J2735 format and then forward them through the internal Apache Kafka messaging core so the RBS RSU Immediate Forward Plug-in can relay the outgoing message to the RSU radio. One execution of this Container App proxies exactly one specific HRI. Thus, a deployment of this Container must exist for each RBS that receives its RTCM data from the CBS. Because the CORS network is only accessible through local State DOT subscriptions, this App must be pre-configured for subscriptions in every State that an RSU is deployed. The currently supported States include Florida, Michigan, Missouri, and Ohio.

rcvw-cbs-api-azure

This Azure Container App is used as a quick way for subscribers to obtain details regarding the rail crossing itself. It is currently populated by the FRA database (currently for Florida, Michigan, and Ohio), with tables and fields that should match that schema. Built using a DAB container, however, allows for web-based queries of the data. The URL for CBS API is:

<https://rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings>

This location supports RESTful query options as described in the following Microsoft Azure help topic:

<https://learn.microsoft.com/en-us/azure/data-api-builder/rest>

Without s provided login ID, a permission denied error will be returned. Therefore, these examples utilize a test login for demonstration purposes. The “demo” login may be removed at a later date, so specific requests should be made for future access.

For example, to list all the equipped HRIs in a JSON format, query the page with:

[https://rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?\\$filter=HRI_ID_gt_0&uuid=demo](https://rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?$filter=HRI_ID_gt_0&uuid=demo)

Or, by FRA crossing identifier:

rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings/CrossingID/867751N?uuid=demo

To list all the HRIs in a given state:

[rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?\\$filter=StateName eq '%27MI%27&uuid=demo](https://rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?$filter=StateName eq '%27MI%27&uuid=demo)

Note that when returning a lot of records as above, the system will automatically break them into pages for you, sending a `nextLink` value to use for retrieving the next page.

Finally, within a certain (square) latitude and longitude boundary, which works well for the viewable area of a graphical map:

[rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?\\$filter=Latitude gt 41.7 and Latitude lt 41.8 and Longitude lt -83.5 and Longitude gt -83.6&uuid=demo](https://rcvw-cbs-api.lemonmushroom-fb00896d.eastus.azurecontainerapps.io/api/Crossings?$filter=Latitude gt 41.7 and Latitude lt 41.8 and Longitude lt -83.5 and Longitude gt -83.6&uuid=demo)

Note that field selection is also possible in order to reduce the size of the data transfer. Refer to the Azure Data API Builder documentation for more information and other examples.

Azure Data Log

The CBS system logs all the incoming and outgoing Azure Service Bus messages within the standard parameters for configured for the individual topics. This may vary by the volume of those individual data sets. For example, the SPaT messages are coming in from the RBS at 10 times per second, but are decoded and persisted to a data store long term, therefore can be removed immediately after processing or upon a maximum 1 hour expiration. However, HRI Events generated by the CBS should only be removed after a much longer expiration period, such as 1 month. Finally, the CBS Event Manager scheduler queues must never contain more than one entry.

All CBS message logs can be retrieved directly from the Azure Portal, but a subscription must already exist for each topic. Message data for the project was dumped to Excel for persistence. The subset of messages that are written to the Azure SQL database can be queried off-line using the typical SQL tools in order to determine past activations and other useful historical reference. The database schema used for this project is depicted in [Figure 4](#), while helpful pre-defined data views made available are depicted in [Figure 5](#). The RCVW III project code contains an SQL file for constructing both of these.

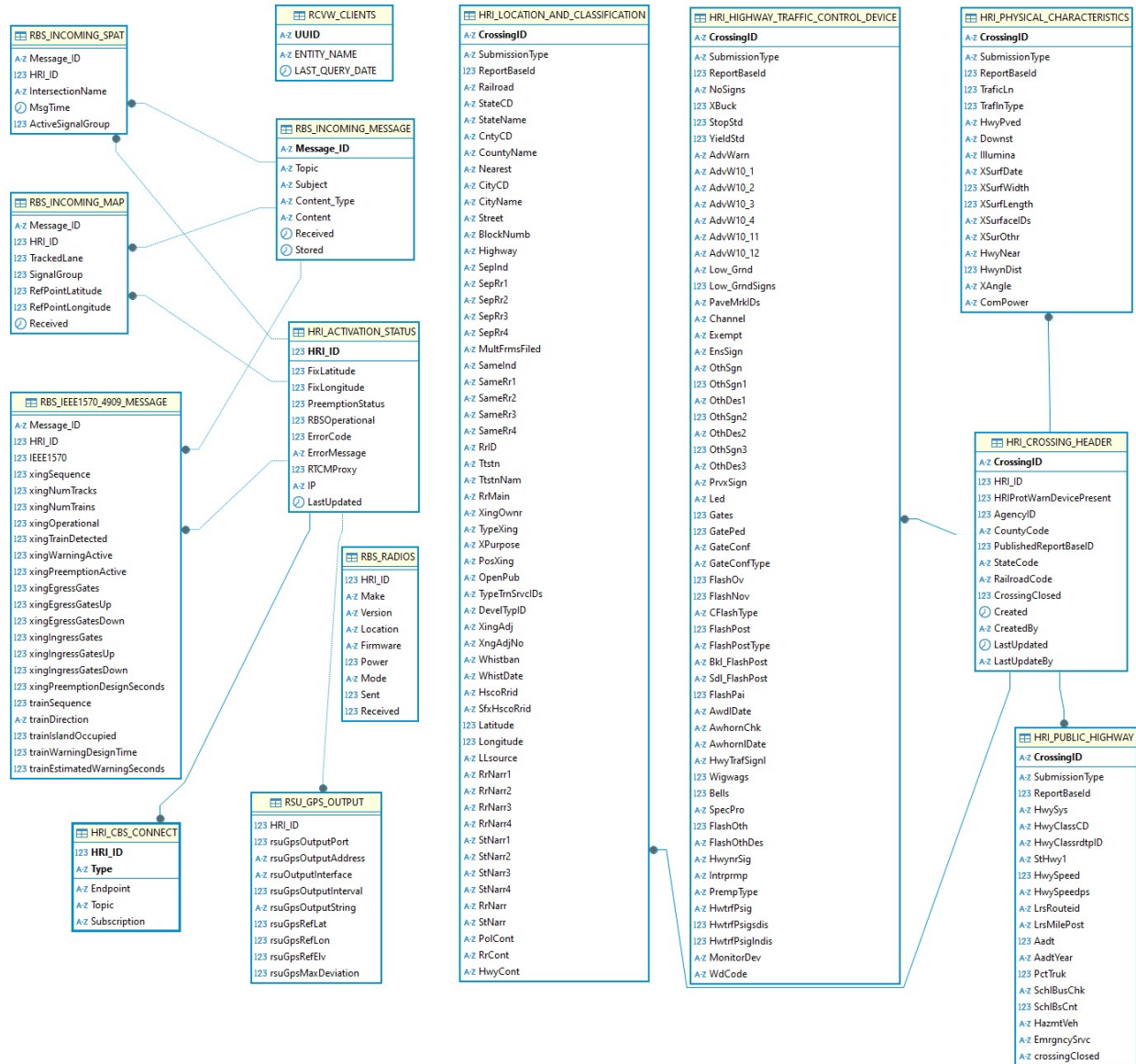


Figure 4 – RCVW CBS Database Schema

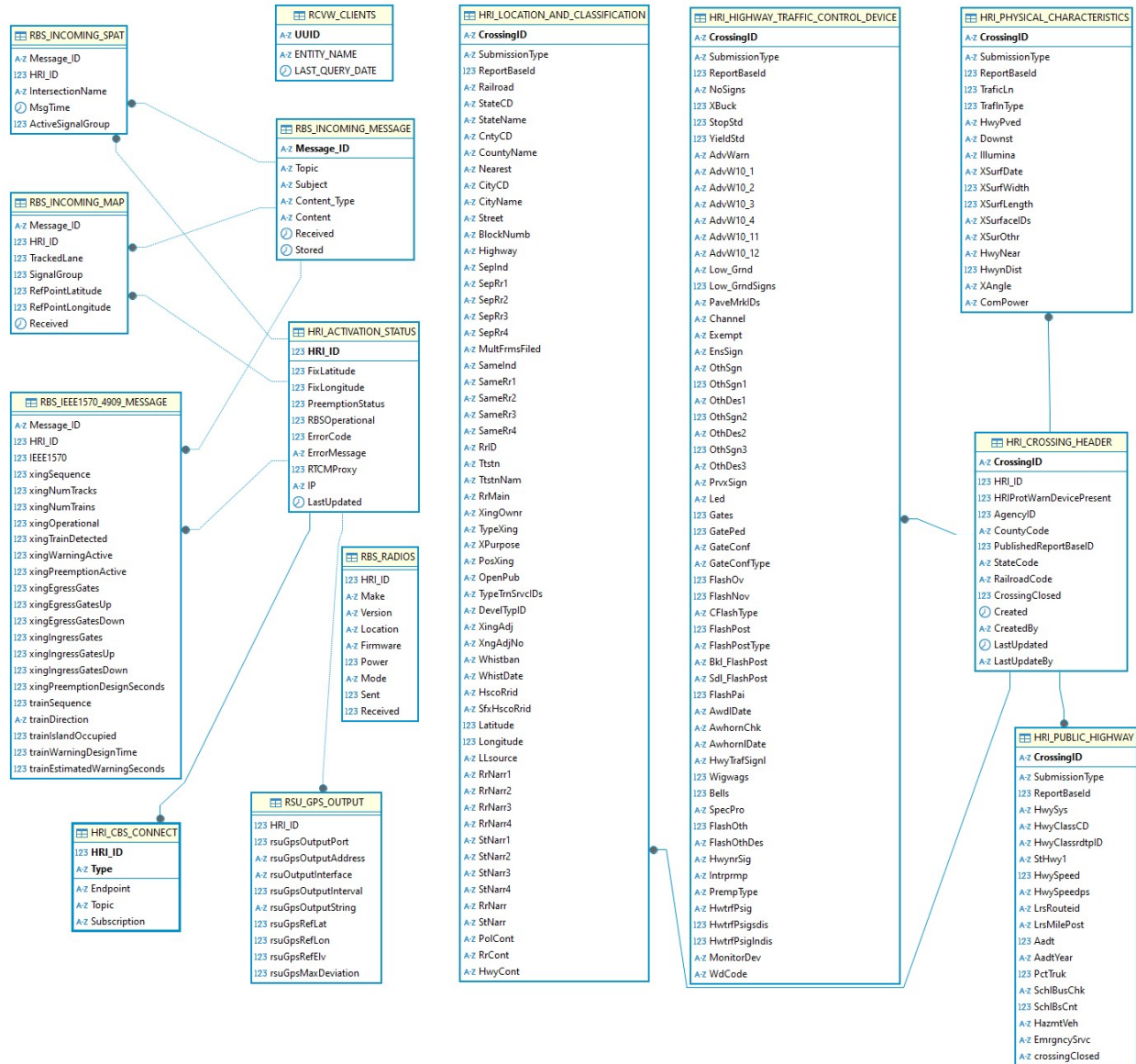


Figure 5 – RCVW CBS Database Views

Chapter 4. Code

The RCVW project code is distributed opensource by Battelle and FRA under the Apache License, Version 2.0. The current release is located under GitHub:

<https://github.com/RCVW-PHASEIII/RCVW>

The file structure includes:

- A `doc` folder where one may find the latest RCVW and V2X-Hub project documentation,
- A `build` folder where one may find some compilation and deployment tools
- A `src` folder where one may find all the C and C++ source code¹⁰

Building

```
$ mkdir RCVW/build/$(dpkg --print-architecture)-Debug
$ cd RCVW/build/$(dpkg --print-architecture)-Debug
$ cmake -DTMX_PROJECT=rcvw ..
$ make package-all
$ sudo make install
```

RBS Component

The Roadside-based Subsystem compiles into a single Docker image.

```
$ make docker-image-rcvw-rbs
```

VBS Component

The Vehicle-based Subsystem compiles into a single Docker image.

```
$ make docker-image-rcvw-vbs
```

CBS Components

The Cloud-based Subsystem has multiple Docker images.

```
$ make docker-image-cbs-api
$ make docker-image-cbs-event-mgr
$ make docker-image-cbs-ingest
```

¹⁰ The Python CBS code is currently stored within the `build` folder

```
$ make docker-image-cbs-rtcm
```

Execution

Each Docker image for an RCVW component comes with an associated docker-compose.yaml file under the build directory to help execute each component.

For example:

```
$ docker compose up -d rcvw-rbs
$ docker compose up -d rcvw-vbs
$ docker compose up -d rcvw-cbs-api
$ docker compose up -d rcvw-cbs-event-mgr
$ docker compose up -d rcvw-cbs-ingest
$ docker compose up -d rcvw-cbs-rtcm
```

While the CBS deployment Docker images contain everything required to operate on any Linux system, the recommended deployment is within an Microsoft Azure subscription. This generally requires that the image be pushed into an available Docker repository, such as Azure Container Registry, and some Container Apps Environment configured with the Function App or Container App resources described above.

Each of these commands require a set of basic environment variables that configures the deployment for execution. An example environment file comes with the source code under the `build` sub-folder that can be used as an instruction guide and template:

```
#####
# The RBS Environment for any Docker-based RCVW deployment
#####
#
# Highway-Rail Grade Crossing Intersection Information
#
# RCVW_MAP: The encoded J2735 MAP structure for the intersection.
#
#         Note that this parameter is required.
RCVW_MAP=
# RCVW_HRI: The identifier for the HRI. Should match the HRI_ID column of a CBS HRI_ACTIVATIONS database table
entry.
#
#         Note that this does NOT (yet) default from the MAP, thus is required. This parameter also is used
as the
#
#         key identifier when RTCM is proxied from the CBS.
RCVW_HRI=
# RCVW_HRI_DESCRIPTION: A quick interseection description to use in the HRI SPaT message.
```

```

#           Note that this does NOT (yet) default from the MAP, thus is required.
RCVW_HRI_DESCRIPTION=
# RCVW_HRI_TRACKED_LANE: The lane identifier in the intersection that denotes the tracked lane signal group.
#           Note that this is an optional parameter that does NOT (yet) default from the MAP,
#           but will default to 0 and will only work with a consistently defined MAP.
RCVW_HRI_TRACKED_LANE=
# RCVW_HRI_TRACKED_LANE: The lane identifier in the intersection that denotes the vehicle ingress lane signal
group.
#           Note that this is an optional parameter that does NOT (yet) default from the MAP,
#           but will default to 1 and will only work with a consistently defined MAP.
RCVW_HRI_VEHICLE_LANE=
#
# RoadSide-Unit (RSU) Radio Information
#
# RCVW_RSU: The IP address of the local RoadSide-Unit (RSU) radio to broadcast J2735 messages from.
#           Note that this is an optional parameter that will default to the hostname "local-rsu"
RCVW_RSU=
# RCVW_RSU_IFM_PORT: The immediate forward port for the local RoadSide-Unit (RSU) radio.
#           Note that this is an optional parameter that will default to the standard IFM port of 1516
RCVW_RSU_IFM_PORT=
# RCVW_RSU_AUTHPROTO: The authentication protocol for an NTCIP (SNMP v3) connection to the local RoadSide-Unit
(RSU) radio.
#           Note that this is an optional parameter that will default to the standard "SHA" protocol
RCVW_RSU_AUTHPROTO=
# RCVW_RSU_AUTHPASS: The authentication passwod for an NTCIP (SNMP v3) connection to the local RoadSide-Unit
(RSU) radio.
#           Note that this is a required parameter.
RCVW_RSU_AUTHPASS=
# RCVW_RSU_AUTHPROTO: The privacy protocol for an NTCIP (SNMP v3) connection to the local RoadSide-Unit (RSU)
radio.
#           Note that this is an optional parameter that will default to the standard "AES"
encryption
RCVW_RSU_PRIVPROTO=
# RCVW_RSU_AUTHPASS: The authentication passwod for an NTCIP (SNMP v3) connection to the local RoadSide-Unit
(RSU) radio.
#           Note that this is a required parameter.
RCVW_RSU_PRIVPASS=
#
# Highway-Rail Grade Crossing Signal Preemption Detection Information

```

```

#
# RCVS_IEEE_INPUT: The serial device file to use to read in the IEEE 1570 messages.
#
#           Note that this is an optional parameter that forces the RBS to receive the HRI preemption
#           status from the IEEE 1570 messages transmitted over the RS-232 or RS-485 serial device.
#
#           If not set, the system defaults to reading a hard-wired preemption signal from the
#           local Digital I/O port. The latter currently presumes specific Neosys POC hardware.
RCVW_IEEE_INPUT=
# RCVW_DIO_PIN: The pin from the local Digital I/O port to read for an active-low preemption signal.
#
#           Note that this is an optional parameter that will default to 0, and if used currently
#           presumes specific Neosys POC hardware.
RCVW_DIO_PIN=
#
# Highway-Rail Grade Crossing Intersection Ground-Based Correction Information for RealTime Kinematics (RTK)
#
# RCVW_GPS: The GPS device file to configure as a local basestation.
#
#           Note that this is an optional parameter that is only used for a basestation RBS configuration and
#           will default to the gpsd port on localhost.
RCVW_GPSD_SOURCE=
# RCVW_HRI_LAT: The absolute latitude for the intersection.
#
#           Note that this is used only for a basestation RBS configuration or an NTRIP source, however,
#           for a basestation configuration this must be specified in 10^7 degrees, e.g., 399570248,
#           but otherwise must be in decimal degrees, e.g., 39.9570248
RCVW_HRI_LAT=
# RCVW_HRI_LON: The absolute longitude for the intersection.
#
#           Note that this is used only for a basestation RBS configuration or an NTRIP source, however,
#           for a basestation configuration this must be specified in 10^7 degrees, e.g., -832478194,
#           but otherwise must be in decimal degrees, e.g., -83.2478194
RCVW_HRI_LON=
# RCVW_HRI_ALT: The absolute altitude for the intersection.
#
#           Note that this is only used in the basestation RBS configuration and
#           must be in cm above sea level, e.g. 2482
RCVW_HRI_ALT=
# RCVW_NTRIP_SOURCE: The NTRIP caster URL, e.g., my.example.com:2101/mountpoint
#
#           Note that this is an optional parameter only used for a NTRIP source.
RCVW_NTRIP_SOURCE=
#
# Other Runtime Configuration Parameters
#
# RCVW_RBS_IMAGE: The location of the RBS Docker image to use.

```

```

#           Note that this is an optional parameter that will default to the local Docker repository
RCVW_RBS_IMAGE=

# RCVW_RBS_VERSION: The version of the RBS Docker image to use.

#           Note that this is an optional parameter that will default to the current version of the
release (4.0.0)
RCVW_RBS_VERSION=

# RCVW_RBS_MAP_DEBUG: The debug level for the MAP plugin.

#           Note that this is an optional parameter that will default to "DEBUG"
RCVW_RBS_MAP_DEBUG=

# RCVW_RBS_HRI_DEBUG: The debug level for the HRI status plugin.

#           Note that this is an optional parameter that will default to "DEBUG"
RCVW_RBS_HRI_DEBUG=

# RCVW_RBS_RSUIFM_DEBUG: The debug level for the RSU immediate forward plugin.

#           Note that this is an optional parameter that will default to "DEBUG"
RCVW_RBS_RSUIFM_DEBUG=

# RCVW_RBS_RTCM_DEBUG: The debug level for the RTCM plugin.

#           Note that this is an optional parameter that will default to "DEBUG"
RCVW_RBS_RTCM_DEBUG=

#####

#####

# The CBS Environment for any Docker-based RCVW deployment
#####

#####

#

# The Cloud Messaging Service Connection Information

#

# RCVW_CBS_HOST: The hostname for the CBS messaging service, e.g., Azure Service Bus (AMQP).

#           Note that this is an optional parameter that will default to the local AMQP Docker service.
RCVW_CBS_HOST=

# RCVW_CBS_USER: The user or access key name for the CBS messaging service, e.g., Azure Service Bus (AMQP).

#           Note that this is a required parameter.
RCVW_CBS_USER=

# RCVW_CBS_PASS: The password or access key for the CBS messaging service, e.g., Azure Service Bus (AMQP).

#           Note that this is a required parameter/
RCVW_CBS_PASS=

#

# The Cloud Database Storage Connection Information

#

```

```
# RCVW_DB_HOST: The hostname for the CBS database storage component, e.g., Azure SQL.

#           Note that this is an optional parameter that will default to the local SQLite database service
RCVW_DB_HOST=

# RCVW_DB_NAME: The name of the database for the CBS database storage component, e.g., Azure SQL.

#           Note that this is an optional parameter that will default to "rcvw-db"
RCVW_DB_NAME=

# RCVW_DB_USER: The user or access key name for the CBS database storage component, e.g., Azure SQL.

#           Note that this is a required parameter.
RCVW_DB_USER=

# RCVW_DB_USER: The password or access key for the CBS database storage component, e.g., Azure SQL.

#           Note that this is a required parameter.
RCVW_DB_PASS=

#

# The Cloud Database API Connection Information

#

# RCVW_CBS_API: The hostname for the CBS API service, e.g., Azure Data API Builder (DAB).

#           Note that this is an optional parameter that will default to the local DAB Docker service.
# RCVW_CBS_API='dab-container-app.redwave-53404ddl.eastus.azurecontainerapps.io'

#

# Other Runtime Configuration Parameters

#

# RCVW_CBS_TARGET: The target CBS implementation for the Docker containers, e.g., Microsoft Azure

#           Note that this is an optional parameter that will default to "azure"
RCVW_CBS_TARGET=

# RCVW_CBS_API_IMAGE: The location of the CBS API Docker image to use.

#           Note that this is an optional parameter that will default to the local Docker repository
RCVW_CBS_API_IMAGE=

# RCVW_CBS_API_VERSION: The version of the CBS API Docker image to use.

#           Note that this is an optional parameter that will default to the current version of the
release (4.0.0)
RCVW_CBS_API_VERSION=

# RCVW_CBS_INGEST_IMAGE: The location of the CBS database ingest Docker image to use.

#           Note that this is an optional parameter that will default to the local Docker
repository
RCVW_CBS_INGEST_IMAGE=

# RCVW_CBS_INGEST_VERSION: The version of the CBS database ingest Docker image to use.

#           Note that this is an optional parameter that will default to the current version of
the release (4.0.0)
RCVW_CBS_INGEST_VERSION=
```

```

# RCVW_CBS_RTCM_IMAGE: The location of the CBS RTCM proxy Docker image to use.

#           Note that this is an optional parameter that will default to the local Docker repository
RCVW_CBS_RTCM_IMAGE=

# RCVW_CBS_RTCM_VERSION: The version of the CBS RTCM proxy Docker image to use.

#           Note that this is an optional parameter that will default to the current version of
the release (4.0.0)
RCVW_CBS_RTCM_VERSION=

# RCVW_CBS_EVENTMGR_IMAGE: The location of the CBS event manager Docker image to use.

#           Note that this is an optional parameter that will default to the local Docker
repository
RCVW_CBS_EVENTMGR_IMAGE=

# RCVW_CBS_EVENTMGR_VERSION: The version of the CBS event manager Docker image to use.

#           Note that this is an optional parameter that will default to the current version
of the release (4.0.0)
RCVW_CBS_EVENTMGR_VERSION=

#####
#####

# The VBS Environment for any Docker-based RCVW deployment
#####
#####

#

# Vehicle Configuration Data

#

# RCVW_GPS: The device file for the local GNSS receiver.

#           Note that this is a optional parameter that defaults to /dev/ttyGPS.

#           See https://gist.github.com/edrol5/1c6cd63894836ed982a7d88bef26e4af for details on
#           how to configure your udev to link the local GPS device to /dev/ttyGPS.
RCVW_GPS=

# RCVW_VBS_VEHICLE: An identifier for this vehicle to use in the log messages

#           Note that this is an optional parameter that defaults to 0
RCVW_VBS_VEHICLE=

# RCVW_VBS_ANTENNA_X: The position of the antenna (in meters) from the driver side of the vehicle.

#           Note that this is a required parameter.
RCVW_VBS_ANTENNA_X=

# RCVW_VBS_ANTENNA_Y: The position of the antenna (in meters) from the front of the vehicle.

#           Note that this is a required parameter.
RCVW_VBS_ANTENNA_Y=

# RCVW_VBS_ANTENNA_HEIGHT: The position of the antenna (in meters) above the road surface.

```

```

#                               Note that this is a required parameter.

RCVW_VBS_ANTENNA_HEIGHT=

# RCVW_VBS_TYPE: The type of vehicle that is being used, either (1) for a car, (2) for a light truck, (3) for a
heavy truck

#                               This is an optional parameter that will default to (1) for a car.

RCVW_VBS_TYPE=

# RCVW_VBS_LENGTH: The length of vehicle (in meters) that is being used, from front to back.

#                               This is an optional parameter that will default "4.8", however a misrepresented
#                               vehicle length will impact performance of the on-track warnings.

RCVW_VBS_LENGTH=

#

# Other Runtime Configuration Parameters

#

# RCVW_VBS_IMAGE: The location of the VBS Docker image to use.

#                               Note that this is an optional parameter that will default to the local Docker repository

RCVW_VBS_IMAGE=

# RCVW_VBS_VERSION: The version of the VBS Docker image to use.

#                               Note that this is an optional parameter that will default to the current version of the
release (4.0.0)

RCVW_VBS_VERSION=

# RCVW_OBU_FWD_PORT: The local port to listen for incoming J2735 messages from the on-board unit (OBU) radio

#                               Note that this is an optional parameter that will default to "26789".

RCVW_OBU_FWD_PORT=

#####

#####

# Calculated Environment Variables for any Docker-based RCVW deployment. DO NOT EDIT

#####

#####

RCVW_CBS="${RCVW_CBS_HOST:+sb://${RCVW_CBS_USER:?Missing CBS message service user}:${RCVW_CBS_PASS:?Missing CBS
messaging service key}@${RCVW_CBS_HOST:-cbs-rabbitmq}}"

```

Appendix A: CBS Resources

Resource Name	Resource Type	Description
<code>rcvw-db-server</code>	SQL Server	<code>rcvw-db-server.database.windows.net</code>
<code>rcvw-db</code>	SQL Database	Contains the tables for the CBS Data Log
<code>rcvwcbssb</code>	Service Bus Namespace	Holds the topics and subscriptions for the RCVW CBS Azure Service Bus
<code>cbs.scheduler.hri-check</code>	Queue	DO NOT USE This topic is used internally to trigger the event manager checks.
<code>cbs.scheduler.rbs-check</code>	Queue	DO NOT USE This topic is used internally to trigger the event manager checks.
<code>cbs.events</code>	Topic	The topic for all outgoing HRI event messages. The <code>cbs-outgoing</code> subscription can be used to receive all HRI event messages, with the message <code>subject</code> indicating the HRI ID that the event pertains to. Additionally, a unique subscription within this topic will exist for each HRI ID an easy filter for events specific to that crossing.
<code>j2735.map</code>	Topic	The rsu-incoming subscription holds all incoming J2735 MAP messages from each RBS. Used to trigger the cloud ingest operations.
<code>j2735.spat</code>	Topic	The rsu-incoming subscription holds all incoming J2735 SPaT messages from each RBS. Used to trigger the cloud ingest operations.
<code>j2735.rtcmm</code>	Topic	The rsu-incoming subscription holds all incoming J2735 RTCM messages from each RBS. Used to trigger the cloud ingest operations.
<code>tmx.plugin.hristatus.error</code>	Topic	The rsu-incoming subscription holds all incoming error messages from each RBS HRI Status Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.hristatus.status</code>	Topic	The rsu-incoming subscription holds all incoming diagnostic and status messages from each RBS HRI Status Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.map.error</code>	Topic	The rsu-incoming subscription holds all incoming error messages from each RBS MAP Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.map.status</code>	Topic	The rsu-incoming subscription holds all incoming diagnostic and status messages from each MAP Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.rsuimmediateforward.error</code>	Topic	The rsu-incoming subscription holds all incoming error messages from each RBS RSU Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.rsuimmediateforward.status</code>	Topic	The rsu-incoming subscription holds all incoming diagnostic and status messages from each RBS RSU Plug-in. Used to trigger the cloud ingest operations.

<code>tmx.plugin.rtcml.error</code>	Topic	The rsu-incoming subscription holds all incoming error messages from each RBS RSU Plug-in. Used to trigger the cloud ingest operations.
<code>tmx.plugin.rtcml.status</code>	Topic	The rsu-incoming subscription holds all incoming diagnostic and status messages from each RBS RSU Plug-in. Used to trigger the cloud ingest operations.
<code>v2x.rtcml3</code>	Topic	The topic for all outgoing RTCM 3.3 event messages. The <code>cbs-outgoing</code> subscription can be used to receive all RTCM messages, with the message <code>subject</code> indicating the HRI ID that the event pertains to. Additionally, a unique subscription within this topic will exist for each HRI ID an easy filter for events specific to that crossing.
<code>rcvw-cbs-api</code>	Container App	The RESTful API server for the CBS Data Log. Uses Azure Data API Builder https://dab-container-app.redwave-53404dd1.eastus.azurecontainerapps.io/api
<code>rcvw-cbs-event-mgr</code>	Function App	A function that detects and relays HRI events. It is triggered periodically when there is an HRI to manage.
<code>rcvw-cbs-ingest</code>	Function App	A function that writes incoming messages to the rcvw-db database. Note that this app is developed as a Function App but is actually deployed as a container since it contains custom C++ code used to decode the J2735 messages.
<code>rcvw-cbs-rtcml</code>	Container App	The proxy RTCM server for the RCVW RBS systems. Only connects when there is an HRI to manage.
<code>rcvw-cbs-containers</code>	Container App Environment	The common Azure environment to deploy the Container Apps to.
<code>rcvw-cbs-functions</code>	Function App Environment	The common Azure environment to deploy the Function Apps to.